

# MODULUS

## Retail Operations Platform

### System Design Document

Version 1.0

---

# 1. Overview

---

Modulus is a micro-frontend retail operations platform designed to simulate the internal tooling used by operations teams at large retailers. It is not a customer-facing application. It is the back-office system through which staff manage products, process orders, and monitor business performance.

The platform is decomposed into four independently deployable frontend applications, each owned by a separate team and each handling a distinct business domain. This decomposition reflects the organisational and technical constraints that justify micro-frontend architecture in production environments at scale.

The engineering goal is to demonstrate that the four applications can be composed into a single coherent user experience, share authentication state and design primitives, and yet deploy completely independently. A change in the Orders application should never require a redeployment of the Inventory or Analytics applications.

## 2. System Context

---

### 2.1 The Fictional Organisation

Modulus Retail is a mid-to-large fictional retailer with three distinct operations teams. Each team owns a separate business domain and has independent release cadences.

| Team           | Domain                          | Application       | Deployment URL               |
|----------------|---------------------------------|-------------------|------------------------------|
| Shell Team     | Platform infrastructure         | modulus-shell     | modulus-shell.vercel.app     |
| Inventory Team | Product catalogue and stock     | modulus-inventory | modulus-inventory.vercel.app |
| Orders Team    | Order processing and fulfilment | modulus-orders    | modulus-orders.vercel.app    |
| Analytics Team | Business intelligence           | modulus-analytics | modulus-analytics.vercel.app |

### 2.2 What the Product Is Not

- A customer-facing storefront or e-commerce site
- A system with real backend infrastructure or a live database

- A mobile application or PWA
- A multi-tenant SaaS product

Authentication is handled via a mock JWT system with hardcoded credentials. Data is served from a lightweight mock API layer using Mock Service Worker (MSW). These decisions are deliberate: the portfolio objective is to demonstrate frontend architecture, not backend engineering.

## 3. Architecture

---

### 3.1 Micro-Frontend Topology

The architecture follows the Host-Remote pattern of Webpack Module Federation. The shell application acts as the host. The three feature applications act as remotes. No remote loads another remote. No remote knows about other remotes. All composition happens in the shell.

The following diagram represents the runtime composition topology:

```
Browser
|
+-- modulus-shell (Host)
    |-- Loads: AuthContext, Layout, Router
    |-- Route /inventory --> loads modulus-inventory Remote
    |-- Route /orders    --> loads modulus-orders Remote
    +-- Route /analytics --> loads modulus-analytics Remote
```

Each remote is loaded asynchronously at runtime from its own deployed Vercel URL. The shell does not bundle remote code at build time. If a remote is updated and redeployed, the shell picks up the new version on the next navigation to that route without requiring its own redeployment.

### 3.2 Module Federation Configuration

#### Shell (Host) Configuration

The shell declares no exposes. It consumes all three remotes via remotes configuration:

```
// webpack.config.js (shell)
new ModuleFederationPlugin({
  name: "shell",
  remotes: {
```

```

    inventory: "inventory@[INVENTORY_URL]/remoteEntry.js",
    orders:    "orders@[ORDERS_URL]/remoteEntry.js",
    analytics: "analytics@[ANALYTICS_URL]/remoteEntry.js",
  },
  shared: {
    react:      { singleton: true, requiredVersion: "^18.0.0" },
    "react-dom": { singleton: true, requiredVersion: "^18.0.0" },
  },
})

```

## Remote Configuration (per MFE)

Each remote exposes its root App component and declares React as a shared singleton:

```

// webpack.config.js (inventory, orders, analytics)
new ModuleFederationPlugin({
  name: "inventory", // unique per remote
  filename: "remoteEntry.js", // standard entry point
  exposes: {
    "./App": "./src/App", // root component exposed
  },
  shared: {
    react:      { singleton: true, requiredVersion: "^18.0.0" },
    "react-dom": { singleton: true, requiredVersion: "^18.0.0" },
  },
})

```

## 3.3 Shared Singleton React Instance

React must run as a single instance across all four applications. Duplicate React instances cause hook violations and context isolation failures. The `singleton: true` flag in the shared configuration enforces this: whichever application loads first provides the React instance, and all subsequent applications reuse it.

If React version constraints conflict between applications, Module Federation throws a runtime error. All four applications must declare the same React version range in their shared configuration.

## 3.4 Monorepo Structure

All four applications and shared packages live in a single monorepo managed by Turborepo. This enables shared TypeScript configuration, lint rules, and design tokens while maintaining independent build and deploy pipelines per application.

```
modulus/
  apps/
    shell/          # Host application
    inventory/      # Inventory MFE
    orders/         # Orders MFE
    analytics/      # Analytics MFE
  packages/
    ui/            # Shared component library
    auth/          # Shared auth context and hooks
    types/         # Shared TypeScript types
    config/        # Shared tsconfig, eslint, tailwind
  .github/
    workflows/     # CI/CD pipeline definitions
  playwright/      # E2E test suite
  turbo.json       # Turborepo pipeline config
  package.json     # Root workspace config
```

## 4. Application Specifications

---

### 4.1 Shell — modulus-shell

The shell is the host application. It is the only application the user navigates to directly. It owns the following concerns:

- Authentication: login page, JWT token storage, route protection
- Persistent layout: top navigation bar with user profile, sidebar with section links
- Routing: maps URL paths to remote MFE components
- Shared context: provides AuthContext to all remotes via React context propagation
- Error boundaries: catches remote load failures and renders fallback UI
- Loading states: skeleton placeholder while remote module loads

The shell does not contain any business logic related to inventory, orders, or analytics. If a remote fails to load, the shell renders an isolated error state for that section without affecting the rest of the platform.

### 4.2 Inventory MFE — modulus-inventory

The Inventory MFE is owned by Team Inventory and is responsible for the complete product catalogue management surface. Features:

- Paginated product table with search by name or SKU
- Filter by category (Electronics, Apparel, Home, Sports, Other)
- Stock level indicator with colour coding: green (> 20 units), amber (5 to 20 units), red (< 5 units)
- Add new product form with fields: name, SKU, category, price, stock quantity, image URL
- Edit product via slide-over panel
- Deactivate and reactivate product listings
- Category management: add and rename categories

All data operations in the Inventory MFE communicate with the mock API at the `/api/products` and `/api/categories` endpoints defined in the API Contract Document.

### 4.3 Orders MFE — `modulus-orders`

The Orders MFE is owned by Team Orders and handles order management and customer lookup. Features:

- Order table filterable by status: pending, processing, shipped, delivered, cancelled
- Search by order ID or customer email
- Order detail view with customer information, line items, and totals
- Status update workflow with a confirmation step before committing
- Order timeline showing the full status history with timestamps
- Customer lookup returning all orders for a given email

Status transitions follow a defined state machine. Not all transitions are valid. The valid transitions are: pending to processing, processing to shipped, shipped to delivered, and any status to cancelled. The UI enforces these constraints.

### 4.4 Analytics MFE — `modulus-analytics`

The Analytics MFE is owned by Team Analytics and is a read-only business intelligence dashboard. Features:

- Daily revenue chart for the last 30 days (Recharts LineChart)
- Order volume by status (Recharts PieChart with labels)
- Top 10 selling products by revenue (Recharts BarChart, horizontal)
- Regional sales breakdown (Recharts BarChart, horizontal)
- Metric cards: total revenue, total orders, average order value, return rate
- Date range selector affecting all charts simultaneously

The Analytics MFE does not write data. It only reads from the /api/analytics endpoints. All chart data is pre-aggregated on the mock API side.

## 5. Shared Packages

---

### 5.1 packages/ui

A shared component library consumed by all four applications. Components are built with React and styled with Tailwind CSS using a shared design token set. The library is not published to npm. It is consumed directly within the monorepo via workspace references.

| Component               | Description                                                            |
|-------------------------|------------------------------------------------------------------------|
| Button                  | Primary, secondary, destructive, and ghost variants with loading state |
| Table                   | Sortable, paginated data table with configurable column definitions    |
| Badge                   | Status indicator with colour variants mapped to domain statuses        |
| Modal                   | Accessible dialog with focus trap and keyboard dismissal               |
| SlideOver               | Side panel for edit forms, slides in from the right                    |
| Skeleton                | Loading placeholder matching the shape of common content blocks        |
| Card                    | Content container with header, body, and optional footer               |
| Input, Select, Textarea | Controlled form inputs with error state support                        |
| Toast                   | Non-blocking notification with success, error, and info variants       |
| EmptyState              | Illustrated empty state with configurable message and action           |

### 5.2 packages/auth

Exports an AuthContext provider and a useAuth hook. The shell wraps the entire application with the AuthContext provider. Each remote consumes useAuth to access the current user and token.

Because React runs as a singleton, the context provided by the shell is accessible inside remote components without any special configuration. The remote simply calls useAuth() and receives the same context instance the shell created.

```
// Usage inside any remote
import { useAuth } from "@modulus/auth";
```

```
const { user, token, logout } = useAuth();
```

## 5.3 packages/types

Shared TypeScript type definitions consumed by all applications and the mock API. Ensures type contracts are consistent across application boundaries. Key types:

```
type Product = {  
  id: string;  
  name: string;  
  sku: string;  
  category: string;  
  price: number;  
  stock: number;  
  active: boolean;  
  imageUrl?: string;  
};
```

```
type Order = {  
  id: string;  
  customerId: string;  
  customerEmail: string;  
  status: OrderStatus;  
  items: OrderItem[];  
  total: number;  
  createdAt: string;  
  timeline: StatusEvent[];  
};
```

```
type OrderStatus = "pending" | "processing" | "shipped" | "delivered" |  
"cancelled";
```

## 5.4 packages/config

Shared configuration files referenced by all applications via extends:

- tsconfig.base.json: strict mode, path aliases, ESNext target
- .eslintrc.base.js: React, TypeScript, and import rules
- tailwind.config.base.js: shared design tokens (colours, spacing, typography)



## 6. Authentication

---

Authentication uses a mock JWT system. There is no external identity provider. On login, the mock API validates credentials against a hardcoded user set and returns a signed JWT. The shell stores the token in memory (not localStorage) and passes it via the AuthContext.

Protected routes in the shell check for a valid token before loading any remote. If the token is absent or expired, the user is redirected to the login page. Remotes trust the token provided by the shell via context and include it in API request headers.

| Credential                          | Role               | Access                                     |
|-------------------------------------|--------------------|--------------------------------------------|
| ops@modulus.com / password123       | Operations Manager | All sections: Inventory, Orders, Analytics |
| inventory@modulus.com / password123 | Inventory Staff    | Inventory only                             |
| orders@modulus.com / password123    | Fulfilment Staff   | Orders only                                |

Role-based access control is enforced in the shell router. Navigating to a section the current role cannot access redirects to an Access Denied page without loading the remote.

## 7. Data Layer

---

All data is served by a Mock Service Worker (MSW) instance running in the browser. MSW intercepts fetch requests and returns seeded JSON responses. There is no backend server, no database, and no network egress required for data operations.

MSW is initialised in the shell and runs in a shared service worker scope. All four applications make fetch calls to relative /api/\* paths, which MSW intercepts regardless of which application made the call.

The seeded data set includes:

- 120 products across 5 categories
- 500 orders in varied statuses, spanning the last 90 days
- 60 customers with associated order histories
- Pre-aggregated analytics data for the last 30 days

MSW handlers are defined in the packages/types package alongside the type definitions, ensuring handler response shapes always match the declared types. If a type changes, the handler must be updated in the same commit.

## 8. Deployment Topology

---

Each application deploys to its own Vercel project. There are four Vercel projects in total. All four projects live under the same Vercel team account.

| Application       | Vercel Project    | Production URL               | Deployed By            |
|-------------------|-------------------|------------------------------|------------------------|
| modulus-shell     | modulus-shell     | modulus-shell.vercel.app     | shell.yml workflow     |
| modulus-inventory | modulus-inventory | modulus-inventory.vercel.app | inventory.yml workflow |
| modulus-orders    | modulus-orders    | modulus-orders.vercel.app    | orders.yml workflow    |
| modulus-analytics | modulus-analytics | modulus-analytics.vercel.app | analytics.yml workflow |

The shell references remote URLs via environment variables, not hardcoded strings. Each Vercel project has a VITE\_INVENTORY\_URL, VITE\_ORDERS\_URL, and VITE\_ANALYTICS\_URL environment variable set to the production URL of the corresponding remote. This allows the shell to be configured to point to staging or production remotes independently.

## 9. Technology Stack

---

| Concern             | Technology          | Rationale                                                                             |
|---------------------|---------------------|---------------------------------------------------------------------------------------|
| Monorepo management | Turborepo           | Incremental builds, task caching, parallel execution across packages                  |
| Module Federation   | Webpack 5           | Mature, production-proven MFE implementation with runtime loading                     |
| Frontend framework  | React 18            | Concurrent features, stable ecosystem, required singleton behaviour                   |
| Language            | TypeScript (strict) | Shared types across application boundaries catch interface mismatches at compile time |
| Styling             | Tailwind CSS        | Shared design token system via config                                                 |

| Concern      | Technology                | Rationale                                                                           |
|--------------|---------------------------|-------------------------------------------------------------------------------------|
|              |                           | extension, no CSS-in-JS runtime overhead                                            |
| Charts       | Recharts                  | React-native, composable, no additional canvas dependencies                         |
| Client state | Zustand                   | Lightweight, isolated per MFE, no global store coupling between apps                |
| Data layer   | MSW (Mock Service Worker) | Browser-native request interception, no backend required, realistic API simulation  |
| Unit testing | Vitest                    | Vite-native, fast, compatible with jsdom for component testing                      |
| E2E testing  | Playwright                | Cross-browser, reliable, supports network interception and parallel execution       |
| CI/CD        | GitHub Actions            | Native to repository, matrix builds, artifact publishing, Slack webhook integration |
| Deployment   | Vercel                    | Zero-config static deploy, preview URLs, environment variable management            |

## 10. Design Principles

---

### 10.1 Team Autonomy

Each team owns their application end to end: source code, build configuration, deployment pipeline, and test coverage. No team requires another team's approval to ship. The shared packages are versioned and treated as internal dependencies. Breaking changes to shared packages require coordination, not permission.

### 10.2 Fault Isolation

A failure in one remote must not bring down the shell or other remotes. Each remote load is wrapped in a React error boundary. If a remote fails to load (network error, deploy failure, version conflict), the shell renders a fallback UI for that section only. The user can continue navigating to other sections.

### 10.3 Independent Deployability

Every application can be built, tested, and deployed without running or depending on the other applications being deployed first. The only coupling between applications is the shared package versions and the runtime URL references in the shell environment variables.

## 10.4 Shared Primitives, Not Shared Logic

Shared packages contain design tokens, UI components, type definitions, and auth utilities. They do not contain business logic. Inventory logic lives only in modulus-inventory. Orders logic lives only in modulus-orders. Shared packages must remain domain-agnostic.